

Studi Kasus : Peta Jawa Timur

Tujuan : Merepresentasikan hubungan antar kota di Jawa Timur Menggunakan struktur data Graph

Menentukan Node dan Edge :

Node (Vertex)

- Menggunakan 7 Kota sebagai Node yaitu : Surabaya, Malang, Probolinggo, Tuban, Kediri, Madiun, dan Lumajang

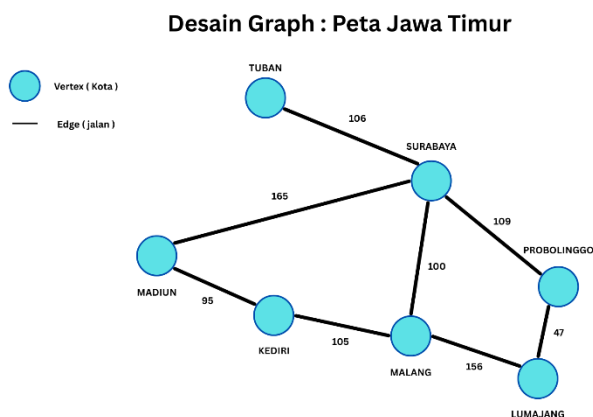
Edge

- Jalan Penghubung Terdekat yang menghubungkan antar Node

Indikator Edge

- Menggunakan Jarak dalam Kilometer (KM) Sebagai Weight

Desain Graph :



Implementasi Pada Python

Adjacencnt List

```
25 # =====
26 # BAGIAN 1: ADJACENCY LIST (menggunakan dictionary)
27 # =====
28
29 def buatAdjacencyList(kota, edges):
30     graph = {k: [] for k in kota}
31     for u, v, w in edges:
32         graph[u].append((v, w)) # simpan (tetangga, bobot)
33         graph[v].append((u, w)) # undirected: dua arah
34     return graph
35
```

PROBLEMS 200 TERMINAL OUTPUT DEBUG CONSOLE SPELL CHECKER ISSUES BY FILE 155 SPELL CHECKER ISSUE

```
=====
ADJACENCY LIST
=====
TUBAN      -> SURABAYA (106 km)
SURABAYA   -> TUBAN (106 km), MADIUN (165 km), MALANG (100 km), PROBOLINGGO (109 km)
MADIUN     -> SURABAYA (165 km), KEDIRI (95 km)
KEDIRI     -> MADIUN (95 km), MALANG (105 km)
MALANG     -> SURABAYA (100 km), KEDIRI (105 km), LUMAJANG (156 km)
PROBOLINGGO -> SURABAYA (109 km), LUMAJANG (47 km)
LUMAJANG   -> MALANG (156 km), PROBOLINGGO (47 km)
```

Adjacency Matrix

```
36 # =====
37 # BAGIAN 2: ADJACENCY MATRIX (menggunakan 2D list)
38 # =====
39
40 def buatAdjacencyMatrix(v, index, edges):
41     # Inisialisasi semua 0 (tidak ada koneksi)
42     mat = [[0] * v for _ in range(v)]
43     for u, v, w in edges:
44         i, j = index[u], index[v]
45         mat[i][j] = w # simpan bobot jarak
46         mat[j][i] = w # undirected: simetris
47     return mat
48
```

PROBLEMS 200 TERMINAL OUTPUT DEBUG CONSOLE SPELL CHECKER ISSUES BY FILE 155

```
=====
ADJACENCY MATRIX (bobot = jarak km, 0 = tidak terhubung)
=====
      TUBA  SURA  MADI  KEDI  MALA  PROB  LUMA
TUBAN    0    106    0     0     0     0     0
SURABAYA 106     0    165     0    100    109     0
MADIUN   0    165     0    95     0     0     0
KEDIRI   0     0    95     0    105     0     0
MALANG   0    100     0    105     0     0    156
PROBOLINGGO 0    109     0     0     0     0     47
LUMAJANG 0     0     0     0    156    47     0
```

Analisis Singkat

1. Jenis Graph

- Undirected Graph (Tidak Berarah): Jalan antara dua kota dianggap berlaku dua arah. Jika terdapat jalan dari Tuban ke Surabaya, maka secara otomatis terdapat jalan dari Surabaya ke Tuban.
- Weighted Graph (Berbobot): Setiap *edge* atau jalan memiliki bobot tertentu yang merepresentasikan jarak (km) antar kota. Contohnya, jalur Surabaya <---> Madiun memiliki bobot sebesar 165 km.

2. Komponen Graph

- Node (Vertex): Merepresentasikan nama-nama kota di Jawa Timur, yaitu: Tuban, Surabaya, Madiun, Kediri, Malang, Probolinggo, dan Lumajang (Total 7 node).
- Edge (Sisi): Merepresentasikan jalan penghubung antar kota tersebut. Terdapat 8 rute jalan utama yang didefinisikan dalam program.

3. Alasan Memilih Edge

- Representasi Infrastruktur Fisik: *Edge* dipilih untuk merepresentasikan jalan raya atau jalur transportasi yang menghubungkan satu kota dengan kota lainnya secara langsung.
- Penyediaan Data Jarak (Bobot): *Edge* digunakan sebagai tempat menyimpan atribut jarak (km), yang sangat krusial untuk menghitung rute terpendek atau estimasi waktu perjalanan antar kota.
- Batasan Aksesibilitas: Pemilihan *edge* tertentu (misalnya hanya 8 rute yang didefinisikan) menunjukkan bahwa hanya jalur-jalur tersebut yang dianggap sebagai akses utama yang tersedia dalam cakupan sistem ini.
- Penentu Jenis Graf: Pemilihan *edge* tanpa arah panah (undirected) dilakukan karena jalan diasumsikan dapat dilalui dari kedua arah dengan beban atau jarak yang sama.
- Efisiensi Pencarian: Dengan menentukan *edge* secara spesifik, algoritma hanya perlu menelusuri jalur yang benar-benar ada (tetangga langsung), sehingga proses *traversal* seperti BFS atau DFS nantinya menjadi lebih efisien.

4. Klasifikasi Graph

Berdasarkan karakteristik data kota dan implementasi kodenya, graph ini diklasifikasikan sebagai:

- Undirected Graph (Graph Tidak Berarah): Hubungan antar kota bersifat dua arah. Dalam kode, hal ini terlihat pada penambahan tetangga yang dilakukan secara timbal balik (misal: jika Tuban terhubung ke Surabaya, maka Surabaya juga terhubung ke Tuban).

- **Weighted Graph (Graph Berbobot):** Setiap *edge* memiliki nilai atau bobot yang merepresentasikan jarak dalam satuan kilometer (\$km\$). Contohnya, rute Surabaya ke Madiun memiliki bobot 165.
- **Sparse Graph (Graph Jarang):** Graph ini dikategorikan sebagai *sparse* karena jumlah *edge* (8) relatif sedikit jika dibandingkan dengan total kemungkinan koneksi maksimum untuk 7 node (yakni $V(V-1)/2 = 21$ koneksi).

5. Representasi yang Paling Cocok

Representasi yang paling cocok untuk studi kasus ini adalah *Adjacency List*. Berikut adalah alasannya:

- **Efisiensi Memori:** Karena graph bersifat *sparse* (sedikit jalur dibanding jumlah kota), *Adjacency List* lebih hemat memori karena hanya menyimpan koneksi yang benar-benar ada. Sebaliknya, *Adjacency Matrix* akan memboroskan ruang dengan menyimpan banyak nilai 0 untuk jalur yang tidak tersedia.
- **Kompleksitas Ruang yang Lebih Baik:** *Adjacency List* hanya membutuhkan ruang sebesar $O(V+E)$, sedangkan *matrix* membutuhkan $O(V^2)$.
- **Efisiensi Iterasi:** Untuk aplikasi peta atau navigasi, kita sering kali perlu mencari "kota tetangga" dari suatu lokasi tertentu. Iterasi tetangga pada *Adjacency List* jauh lebih efisien dibandingkan harus memeriksa seluruh baris pada *matrix*.
- **Kebutuhan Algoritma Traversal:** Representasi ini merupakan dasar yang sangat baik untuk algoritma pencarian rute seperti BFS atau DFS yang akan digunakan untuk mencari jalur terpendek antar kota di pertemuan berikutnya.

6. Penjelasan Node dan Edge

Struktur data graph secara matematis dinyatakan sebagai $G=(V,E)$. Berikut adalah rinciannya:

- **Node (Vertex):** Definisi: Unit dasar atau titik yang merepresentasikan entitas atau objek utama dalam sebuah sistem.
 - Dalam Praktikum 4: Node merepresentasikan nama-nama kota di Jawa Timur, seperti Tuban, Surabaya, Madiun, Kediri, Malang, Probolinggo, dan Lumajang.
- **Edge (Sisi):** Definisi: Garis penghubung yang merepresentasikan hubungan, koneksi, atau jalur antar dua node.
 - Dalam Praktikum 4: Edge merepresentasikan jalan penghubung antar kota. Karena ini adalah *weighted graph*, setiap edge juga menyimpan bobot berupa jarak dalam kilometer (km).

7. Analisis Adjacency List vs Adjacency Matrix

Berdasarkan dokumen teori dan hasil eksekusi program, berikut adalah analisis perbandingannya:

- Adjacency Matrix
 - Bentuk: Menggunakan matriks 2 dimensi berukuran $V \times V$.
 - Kelebihan: Pengecekan apakah ada jalan langsung antar dua kota sangat cepat, yaitu $O(1)$.
 - Mudah dipahami secara visual untuk graph yang kecil.
 - Kekurangan: Boros memori ($O(V^2)$) karena harus menyediakan ruang untuk semua kemungkinan koneksi, meskipun jalan tersebut tidak ada (bernilai 0).
 - Tidak efisien untuk *sparse graph* (graph dengan sedikit jalur).
- Adjacency List
 - Bentuk: Menggunakan daftar tetangga (dalam Python menggunakan dictionary atau list) untuk setiap node.
 - Kelebihan: * Sangat hemat memori karena hanya menyimpan hubungan yang benar-benar ada ($O(V+E)$).
 - Iterasi untuk melihat kota-kota tetangga jauh lebih efisien.
 - Sangat cocok untuk studi kasus peta kota yang sifatnya *sparse*.
 - Kekurangan: * Pengecekan hubungan antara dua kota tertentu lebih lambat dibandingkan matriks.

8. Analisis Representasi (Matrix vs List)

Berdasarkan data yang digunakan, berikut adalah perbandingannya:

- Adjacency List:
 - Kesesuaian: Sangat cocok untuk kasus ini karena termasuk dalam Sparse Graph (graf yang jumlah hubungannya relatif sedikit dibanding total kemungkinan koneksi).
 - Kelebihan: Lebih hemat memori karena hanya menyimpan kota-kota yang benar-benar terhubung. Memudahkan iterasi untuk melihat tetangga langsung dari suatu kota.
- Adjacency Matrix:
 - Kesesuaian: Kurang efisien dalam penggunaan ruang ($O(V^2)$) karena banyak sel yang bernilai 0 (tidak ada jalan langsung).
 - Kelebihan: Sangat cepat ($O(1)$) jika ingin mengecek apakah ada jalan langsung antara dua kota tertentu.

9. Kesimpulan

- Graph sebagai Model Hubungan: Struktur data graph sangat efektif digunakan untuk memodelkan hubungan antar entitas di dunia nyata, seperti keterhubungan antar kota melalui jalur darat.
- Fleksibilitas Representasi: Graph dapat direpresentasikan dengan dua cara utama, yaitu Adjacency Matrix untuk akses pengecekan cepat pada graf yang padat (*dense*), dan Adjacency List untuk efisiensi ruang pada graf yang jarang (*sparse*).
- Efisiensi Adjacency List: Dalam studi kasus peta kota, Adjacency List lebih unggul karena lebih hemat memori ($O(V+E)$) dan mempermudah proses iterasi tetangga yang diperlukan untuk algoritma pencarian rute.
- Karakteristik Data: Penggunaan Weighted Undirected Graph pada praktikum ini memungkinkan penyimpanan informasi yang kompleks, yaitu arah hubungan yang timbal balik (dua arah) serta bobot berupa jarak tempuh antar lokasi.
- Persiapan Traversal: Pemahaman yang mendalam mengenai cara graph disimpan dalam Python (seperti menggunakan *dictionary* atau *list*) menjadi fondasi krusial sebelum mempelajari algoritma pencarian rute lebih lanjut seperti BFS dan DFS.